

CorpPilot: Copiloto Corporativo com IA

Trabalho — IA Generativa Aplicada ao Desenvolvimento

Autor: Bruno Santana

Disciplina: IA Generativa Aplicada ao Desenvolvimento

Data: Julho/2026

Links do projeto:

- Aplicação: <https://corppilot.vercel.app>
 - Repositório: <https://github.com/bsantana/corppilot>
-

1. Contextualização do Problema

A TechCorp é uma empresa de tecnologia em crescimento com centenas de documentos internos espalhados entre planilhas, PDFs, apresentações, procedimentos operacionais, treinamentos e políticas corporativas. Embora essas informações existam, elas não estão facilmente acessíveis para os colaboradores.

Diariamente, profissionais de diferentes áreas perdem tempo procurando respostas para dúvidas recorrentes — como "qual é a política de férias?", "como solicito reembolso?" ou "quais benefícios tenho direito?" — consultando colegas, enviando mensagens em grupos internos ou buscando documentos antigos em pastas compartilhadas.

Esse cenário gera retrabalho, reduz produtividade e dificulta a disseminação do conhecimento dentro da organização. Segundo pesquisas do McKinsey, funcionários gastam em média 1,8 horas por dia buscando informações, representando uma perda significativa de produtividade.

O impacto no negócio é direto: colaboradores que deveriam estar executando tarefas estratégicas ficam presos em buscas operacionais. RH e TI recebem tickets repetitivos sobre políticas já documentadas. Novos funcionários demoram semanas para se orientar em processos internos. A falta de um ponto central de consulta inteligente amplifica esses problemas conforme a empresa cresce.

Com o avanço da Inteligência Artificial Generativa, tornou-se viável construir assistentes que compreendem perguntas em linguagem natural e retornam respostas contextualizadas — transformando documentos estáticos em conhecimento acessível sob demanda.

2. Solução Desenvolvida

O **CorpPilot** é um copiloto corporativo inteligente que permite colaboradores consultarem informações organizacionais através de perguntas em linguagem natural. A solução simula a experiência de assistentes de IA utilizados por grandes empresas como Google (Gemini), Microsoft (Copilot) e Salesforce (Einstein).

Arquitetura

A aplicação foi construída com **Next.js 16** (React + TypeScript) e utiliza a API do **Google Gemini 3.1 Flash-Lite** como motor de IA. A base de conhecimento consiste em 9 documentos markdown organizados por categorias (RH, Financeiro, TI e Operações), hospedada no diretório `docs/` do repositório.

Fluxo de uma pergunta:

1. O colaborador faz uma pergunta no chat
2. O backend carrega os documentos markdown do disco

- 3. Um algoritmo de busca lexical identifica os 4 documentos mais relevantes
- 4. O conteúdo é injetado no system prompt do LLM
- 5. O Gemini gera uma resposta baseada exclusivamente nos documentos
- 6. As fontes consultadas são exibidas como badges na interface

Usuário → Frontend (React) → API /api/chat → knowledge-base.ts → Gemini API → Resposta + Fontes

Funcionalidades Implementadas

Funcionalidade	Descrição
Chat em linguagem natural	Interface de conversa com loading state
Base de conhecimento	9 documentos corporativos fictícios da TechCorp
Citação de fontes	Badges indicando documento e categoria consultados
Histórico de conversas	Persistido no localStorage do browser
Categorias	RH, Financeiro, TI e Operações com perguntas sugeridas
API de resumo	Endpoint /api/summarize para resumir documentos
Deploy em produção	Hospedado na Vercel em corppilot.vercel.app

Demonstração

A aplicação está publicada e acessível em: <https://corppilot.vercel.app>

Screenshots disponíveis em `public/screenshots/` e no repositório GitHub.

3. Ferramentas de IA Utilizadas

Ferramenta	Uso no Projeto	Link
Cursor	Desenvolvimento assistido: geração de componentes, API routes, lógica de negócio, documentação e deploy	cursor.com
Google Gemini API	Funcionalidade principal: chat com contexto de documentos (modelo gemini-3.1-flash-lite)	ai.google.dev
ChatGPT / Claude	Apoio na criação da base de conhecimento e engenharia de prompts	chat.openai.com

Exemplos de Soluções Existentes no Mercado

Glean (glean.com) — Plataforma enterprise de busca com IA que conecta-se a todas as ferramentas da empresa (Google Workspace, Slack, Jira) e permite perguntas em linguagem natural. O CorpPilot segue conceito similar, porém focado em documentos internos de uma empresa fictícia.

Microsoft Copilot — Integrado ao Microsoft 365, permite consultar documentos do SharePoint, e-mails e calendário via linguagem natural. Diferente do CorpPilot por depender do ecossistema Microsoft e exigir licenciamento enterprise.

Notion AI — Assistente embutido no Notion que resume páginas, responde perguntas sobre documentos da workspace e gera conteúdo. O CorpPilot é mais especializado em políticas corporativas com citação obrigatória de fontes.

Critério	CorpPilot	Glean	Notion AI
Foco	Políticas corporativas	Busca enterprise	Produtividade/docs
Citação de fontes	Sim, obrigatória	Sim	Parcial
Custo	API Gemini (free tier)	Enterprise (alto)	Plano pago
Customização	Código aberto	SaaS fechado	SaaS fechado

4. Como a IA Auxiliou na Criação da Aplicação

Geração de Código com Cursor (modo Agent)

A maior parte do código foi gerada com assistência do Cursor em modo Agent. Exemplos concretos:

Componentes React — Prompt: "Crie o componente Chat com input, lista de mensagens e loading state"

- Gerou: Chat.tsx , MessageBubble.tsx , Sidebar.tsx , SourceCitation.tsx

API Routes — Prompt: "Crie API route /api/chat que recebe mensagem e chama Gemini com contexto dos documentos"

- Gerou: src/app/api/chat/route.ts , src/lib/ai.ts , src/lib/knowledge-base.ts

Base de conhecimento — Prompt: "Crie 9 documentos markdown para empresa fictícia TechCorp"

- Gerou: 9 arquivos em docs/rh/ , docs/financeiro/ , docs/ti/ , docs/operacoes/

Documentação — Gerou README.md , docs/SPECS.md , docs/ENTREGA.md e este documento teórico.

Evidências detalhadas em docs/evidencias-ia/DESENVOLVIMENTO.md .

Prints reais do Cursor (evidências visuais)

Durante o desenvolvimento, foram capturados screenshots da interface do Cursor com as conversas reais:

Print	O que mostra
cursor-real-01-visao-geral.png	Visão geral do projeto e funcionalidades geradas
cursor-real-02-migracao-gemini.png	Diff da migração OpenAI → Gemini
cursor-real-03-erro-quota-gemini.png	Erro de quota e modelos descontinuados
cursor-real-04-modelos-disponiveis.png	Pesquisa de modelos na documentação Google
cursor-real-05-bug-historico.png	Correção do bug de histórico do chat
cursor-real-06-fluxo-rag.png	Diagrama do fluxo RAG explicado pela IA

Todos disponíveis em docs/evidencias-ia/ no repositório GitHub.

Migração e Desafios com Modelos de IA

Durante o desenvolvimento, enfrentamos desafios reais com APIs de IA:

- 1. **Migração OpenAI → Gemini** — Inicialmente usamos GPT-4o-mini; migramos para Gemini pelo free tier
- 2. **Modelos descontinuados** — `gemini-2.0-flash` retornou quota zero (descontinuado)
- 3. **Restrição para novos usuários** — `gemini-2.5-flash` retornou 404 para contas novas
- 4. **Solução final** — `gemini-3.1-flash-lite` , recomendado pelo Google para novos projetos

Engenharia de Prompts

O system prompt foi refinado iterativamente:

Versão	Mudança	Resultado
v1	Prompt genérico	Respostas sem fontes, alucinações
v2	Instrução de citar fontes	Fontes aparecem, mas às vezes inventadas
v3	"Responda APENAS com base nos documentos"	Redução significativa de alucinações
v4	Formato estruturado com seção "Fontes:"	Parse automático de fontes no backend

5. Agentes, Automações e Gerenciamento de Contexto

Gerenciamento de Contexto (RAG Simplificado)

O CorpPilot implementa um padrão **Retrieval-Augmented Generation (RAG) simplificado**:

- 1. **System Prompt** — Define persona, regras e documentos relevantes (~4 docs por pergunta)
- 2. **Histórico** — Últimas 6 mensagens da conversa como contexto conversacional
- 3. **Busca de Relevância** — Scoring lexical por tokens (título +3, token +2, conteúdo +1)

O LLM não acessa arquivos diretamente. O backend lê os `.md` , seleciona os relevantes e injeta o texto completo no prompt.

Automações

- Carregamento automático de documentos markdown ao receber cada pergunta
- Parse automático da seção "Fontes:" na resposta da IA
- Persistência automática do histórico no localStorage
- Deploy automático na Vercel a cada push no GitHub

Fluxo Agêntico Simplificado

Embora não utilize um framework de agentes (LangChain, CrewAI), o CorpPilot implementa um fluxo agêntico em 4 etapas:

Receber pergunta → Buscar documentos → Raciocinar (LLM) → Responder com fontes

MCP (Model Context Protocol)

O **Cursor IDE** utiliza MCP para conectar ferramentas externas durante o desenvolvimento (leitura de arquivos, execução de comandos, busca na web). Embora o CorpPilot não implemente um servidor MCP próprio, o protocolo

foi fundamental no processo de desenvolvimento assistido — permitindo que o agente do Cursor lesse o PDF do trabalho, explorasse o código e executasse comandos de build e deploy.

6. Benefícios e Limitações

Benefícios	Limitações
Desenvolvimento 5-10x mais rápido com IA assistida	Dependência de API externa (Google Gemini)
MVP funcional em menos de 1 dia	Busca lexical simples (sem embeddings vetoriais)
Código consistente e bem estruturado	Respostas limitadas aos 9 documentos carregados
Base de conhecimento realista gerada por IA	Sem autenticação de usuários
Deploy gratuito (Vercel + Gemini free tier)	Histórico apenas no browser (localStorage)
Citação de fontes reduz alucinações	Modelos Gemini mudam frequentemente (depreciações)
Documentação gerada automaticamente	Sem suporte a upload de novos documentos pelo usuário
Redução simulada de tempo de busca	API key precisa ser configurada manualmente no deploy

Lições Aprendidas

- Modelos de IA em free tier são instáveis — planejar fallbacks
- Histórico de chat no Gemini exige formato estrito (user/model alternados)
- RAG simplificado funciona bem para poucos documentos (<20)
- IA assistida acelera desenvolvimento, mas revisão humana continua essencial

7. Aspectos Éticos, Responsabilidade e Governança

Privacidade de Dados

- Documentos corporativos são **fictícios** (empresa simulada TechCorp)
- API key armazenada em variáveis de ambiente, nunca exposta ao frontend
- Nenhum dado pessoal real é processado
- Histórico de conversas fica apenas no browser do usuário (localStorage)

Transparência

- Interface indica que respostas são geradas por IA (badge "CorpPilot" + disclaimer)
- Fontes consultadas são exibidas em cada resposta
- Mensagem no rodapé: "CorpPilot utiliza IA para consultar documentos internos"

OWASP Top 10 for LLM Applications

Risco	Mitigação no CorpPilot
LLM01 — Prompt Injection	System prompt com regras rígidas; respostas baseadas apenas em docs
LLM02 — Insecure Output	Respostas renderizadas como texto plano (sem HTML)

LLM06 — Sensitive Info Disclosure	API key protegida server-side (Next.js API Routes)
LLM07 — Insecure Plugin Design	Sem plugins externos; apenas leitura de arquivos locais
LLM09 — Overreliance	Disclaimer orientando verificar com departamentos responsáveis

Governança

- Respostas baseadas exclusivamente em documentos aprovados
 - Instrução explícita no prompt para não inventar informações
 - Recomendação de contato com RH/Financeiro/TI para casos não cobertos
 - Código aberto no GitHub para auditoria
-

8. Referências

- Lovable — <https://lovable.dev>
 - GitHub Copilot — <https://github.com/features/copilot>
 - Model Context Protocol — <https://modelcontextprotocol.io>
 - Google AI Documentation — <https://ai.google.dev>
 - OWASP Top 10 for LLM — <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
 - Glean (solução de mercado) — <https://www.glean.com>
 - Notion AI — <https://www.notion.com/product/ai>
 - Microsoft Copilot — <https://www.microsoft.com/copilot>
 - McKinsey Global Institute — The social economy: Unlocking value and productivity through social technologies
 - Vercel — <https://vercel.com>
 - Next.js — <https://nextjs.org>
-